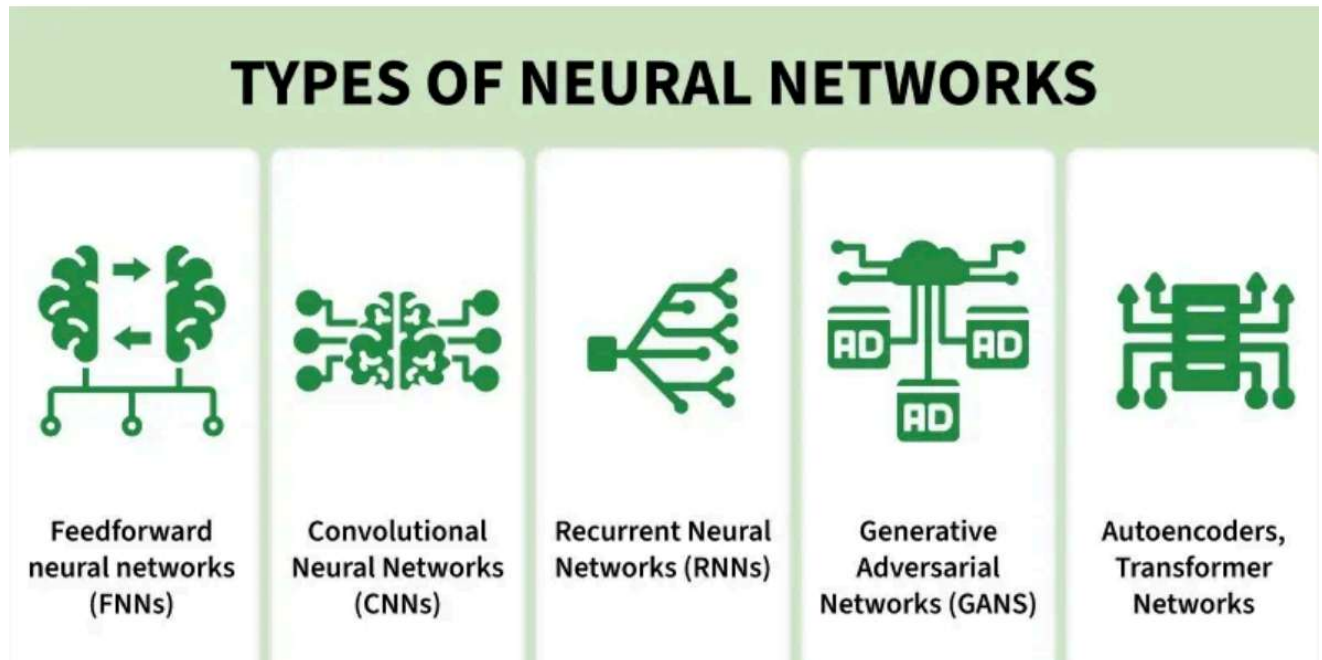


Introduction to Neural Networks

Neural networks power modern AI — from image recognition to large language models. Today you will understand the brain analogy, learn how a network learns step by step, then build your first real neural network in Keras targeting 97%+ accuracy on MNIST. No heavy maths today — intuition and implementation is the goal.

Neural networks are machine learning models that mimic the complex functions of the human brain. These models consist of interconnected nodes or neurons that process data, learn patterns and enable tasks such as pattern recognition and decision-making



Neural networks are capable of learning and identifying patterns directly from data without pre-defined rules. These networks are built from several key components:

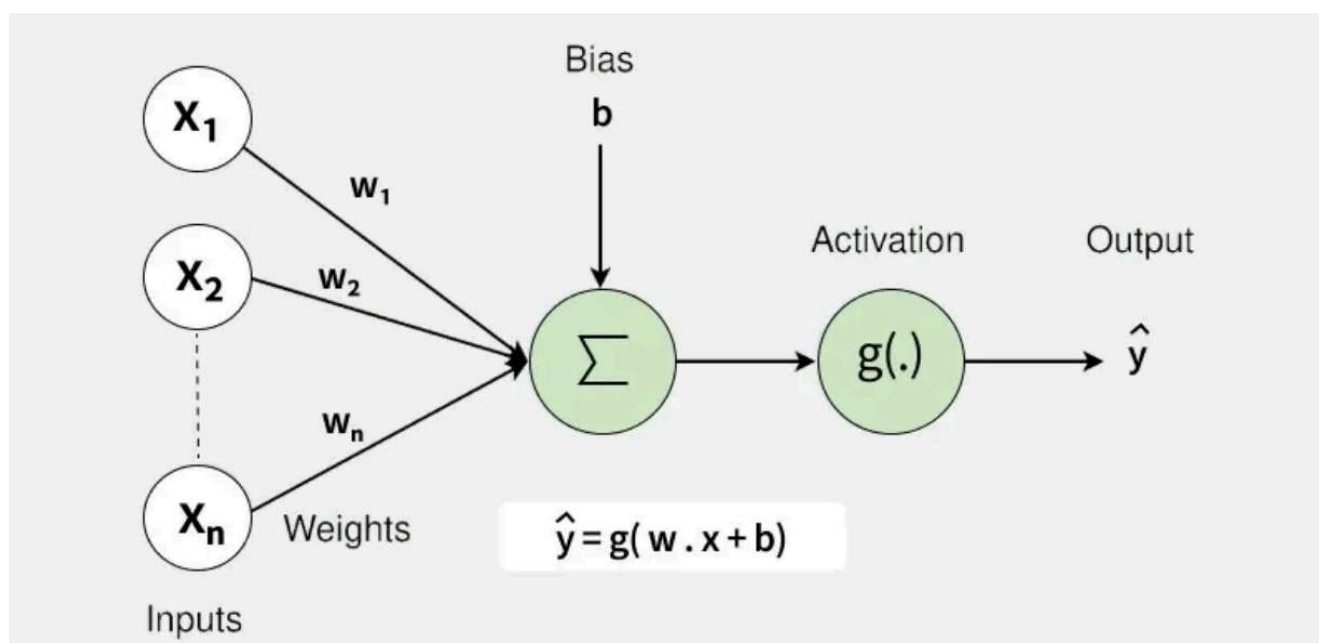
Neurons: The basic units that receive inputs, each neuron is governed by a threshold and an activation function.

Connections: Links between neurons that carry information, regulated by weights and biases.

Weights and Biases: These parameters determine the strength and influence of connections.

Propagation Functions: Mechanisms that help process and transfer data across layers of neurons.

Learning Rule: The method that adjusts weights and biases over time to improve accuracy.



The Human Brain Analogy

Biological Neurons — How Your Brain Works

Your brain contains about 86 billion neurons. Each neuron is a tiny cell that receives electrical signals from other neurons through connections called synapses. When enough signals arrive, the neuron fires — it sends its own signal forward to the next layer of neurons. This chain of firing neurons is how you recognise a face, read these words, or remember a name.

No single neuron understands a face. Thousands of neurons work together — early ones detect edges, middle ones detect shapes, later ones detect eyes and noses. Recognition emerges from cooperation. Artificial neural networks copy this exact idea.

From Neuron to Artificial Neuron

The Artificial Neuron — Step by Step

Step 1 — Receive inputs: Numbers arrive as inputs. For an image, these are pixel values. For a house price model, they might be square footage and location.

Step 2 — Multiply by weights: Each input is multiplied by a weight — a number the network has learned. High weight = this input matters a lot. Low weight = this input barely matters.

Step 3 — Sum everything: Add all weighted inputs together, then add one more learnable number called a bias (shifts the result up or down).

Step 4 — Apply activation function: Pass the sum through a function that squashes it into a useful range and adds non-linearity.

Step 5 — Output: One number leaves this neuron and goes to the next layer.

The Neuron Formula

$$\text{output} = \text{activation}(w_1x_1 + w_2x_2 + \dots + w_nx_n + \text{bias})$$

In compact matrix form (how computers actually compute it):

$$\text{output} = \text{activation}(W \cdot x + b)$$

W is a matrix of weights, x is the input vector, b is the bias vector. The dot product $W \cdot x$ computes all weighted sums at once using matrix multiplication — fast, efficient, and the same on a GPU with millions of neurons.

Weights and Bias — What the Network Actually Learns

When we say a neural network learns, exactly two things change: weights and biases. The architecture (number of layers, neurons per layer) is fixed before training. Training is the process of finding the right values for every weight and bias.

Parameter	What it does	Simple analogy
Weight (w)	How much this input influences the output	Volume knob on each input
Bias (b)	Shifts the activation up or down regardless of input	Thermostat baseline offset
Activation	Non-linear squash function applied after the sum	The neuron's on/off decision

Why Do We Need Activation Functions?

The Non-Linearity Problem

If you chain 10 layers of pure matrix multiplications (no activation functions), the entire network is mathematically equivalent to a single matrix multiplication. Depth gives you nothing. You can replace 10 layers with 1 and get identical results.

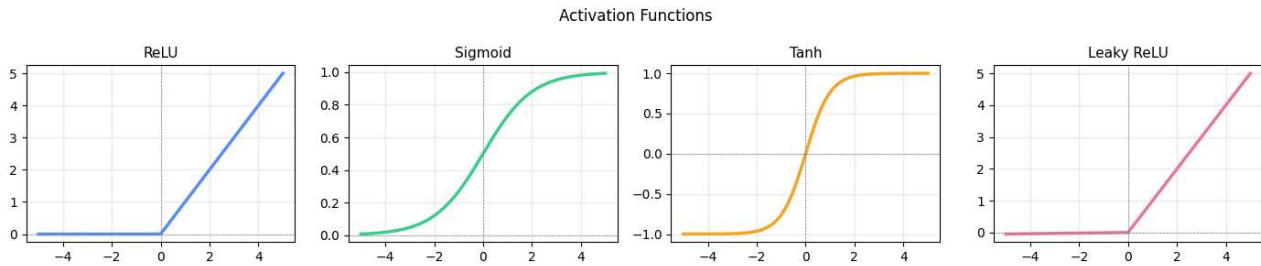
Activation functions break this by introducing non-linearity. With non-linear activations, a network with enough neurons can approximate any continuous function — this is the Universal Approximation Theorem, the mathematical foundation of deep learning.

Activation Functions — Intuition First

Function	Formula	Output Range	Plain English	Use For
ReLU	$\max(0, x)$	0 to ∞	If positive, pass it. If negative, output 0.	Hidden layers — default
Sigmoid	$1 / (1 + e^{-x})$	0 to 1	Squeezes any number into a probability.	Binary output
Softmax	$e^{x_i} / \sum e^{x_j}$	0 to 1 (sums to 1)	Converts scores to class probabilities.	Multi-class output
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	-1 to +1	Like sigmoid but zero-centred.	Hidden layers
Leaky ReLU	$\max(0.01x, x)$	$-\infty$ to $+\infty$	Like ReLU but small leak for negatives.	Fixes dead neurons

ReLU is the default choice for hidden layers today. It is simple, fast, and avoids the vanishing gradient problem that plagued older sigmoid-only networks. Use Softmax on the output layer for multi-class problems — it ensures all class probabilities add up to 1.0.

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(-5, 5, 300)
# Define activation functions
relu = lambda x: np.maximum(0, x)
sigmoid = lambda x: 1 / (1 + np.exp(-x))
tanh_fn = np.tanh
leaky = lambda x: np.where(x > 0, x, 0.01 * x)
fig, axes = plt.subplots(1, 4, figsize=(14, 3))
fns = [relu, sigmoid, tanh_fn, leaky]
names = ['ReLU', 'Sigmoid', 'Tanh', 'Leaky ReLU']
cols = ['#5B8FF9', '#3ECF8E', '#F5A623', '#E879A0']
for ax, fn, name, col in zip(axes, fns, names, cols):
    ax.plot(x, fn(x), color=col, linewidth=2.5)
    ax.axhline(0, color='gray', lw=0.5, ls='--') # x-axis
    ax.axvline(0, color='gray', lw=0.5, ls='--') # y-axis
    ax.set_title(name, fontsize=11)
    ax.grid(alpha=0.2)
plt.suptitle('Activation Functions', fontsize=12)
plt.tight_layout(); plt.show()
```



Building a Neural Network

The Three Types of Layers

Every neural network has the same basic structure — data enters at the left, flows through the middle, and exits at the right:

Layer	What it does	MNIST Example (28×28 digit image)
Input Layer	Holds raw input data — no computation, just receives values	784 neurons — one per pixel in the flattened image
Hidden Layers	Learn patterns — each layer learns increasingly complex features	Layer 1: 128 neurons detect strokes. Layer 2: 64 neurons detect curves
Output Layer	Produces the final prediction — size equals number of classes	10 neurons — one per digit 0 through 9

start with 1-2 hidden layers. Add more only if the model cannot fit the data. More layers = more expressive but harder to train and easier to overfit.

The Forward Pass — Data Flowing Through

Step by Step: Classifying the Digit '7'

Input: The 28×28 image is flattened to 784 numbers (pixel values 0.0–1.0 after normalisation).

Hidden Layer 1 (128 neurons): Each neuron computes $W \cdot x + b$ then applies ReLU. Output: 128 numbers encoding learned features like diagonal strokes and horizontal bars.

Hidden Layer 2 (64 neurons): Receives the 128 features and combines them into 64 higher-level representations — partial shapes, digit-like structures.

Output Layer (10 neurons): Produces 10 raw scores. Softmax converts these to probabilities that sum to 1.0.

Prediction: [0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.01, 0.90, 0.02, 0.01] → highest at index 7 → predicted digit is 7. ✓

Loss Function — Measuring How Wrong We Are

After the forward pass, we compare the prediction to the true label. The loss function converts the error into a single number. The goal of training is to minimise this number.

Loss Function	When to Use	Intuition
<code>sparse_categorical_crossentropy</code>	Multi-class, integer labels (0,1,2...)	Penalises confident wrong answers harshly
<code>categorical_crossentropy</code>	Multi-class, one-hot labels ([0,1,0,0...])	Same maths, different label format
<code>binary_crossentropy</code>	Two classes only (spam/not spam)	Special case of cross-entropy for 2 classes
<code>mean_squared_error</code>	Regression — continuous output	Average squared distance from target

Cross-entropy is large when the model confidently predicts the wrong class, and near zero when it confidently predicts the correct class. This is exactly the behaviour we want — punish arrogant mistakes.

Backpropagation — How the Network Corrects Its Mistakes

Backprop in Plain English

After computing the loss, we need to adjust weights to reduce it. Backpropagation answers: how much did each weight contribute to this mistake?

The Chain Rule: Calculus tells us how to compute the gradient of the loss with respect to every single weight in the network — even weights deep in early layers. We chain the derivatives layer by layer, backward from the output.

The Update: Once we have the gradient for each weight, we nudge it slightly in the direction that reduces the loss: $\text{new_weight} = \text{old_weight} - \text{learning_rate} \times \text{gradient}$

Repeat: Do this for every batch of training examples, every epoch. The network gets a little better each time until loss plateaus.

$$\text{new_weight} = \text{old_weight} - \text{learning_rate} \times \text{gradient}$$

If the gradient is positive (increasing the weight increases the loss), we decrease the weight. If negative, we increase it. The learning rate controls the step size — too large and we overshoot, too small and training takes forever. Adam optimiser handles this automatically.

Gradient Descent Variants

Variant	Description	Pros / Cons
Batch GD	Compute gradient over entire dataset, then update	Stable but impractically slow for large data
Stochastic GD (SGD)	Compute gradient over 1 sample, update immediately	Fast updates but very noisy — oscillates a lot
Mini-batch GD	Compute gradient over small batch (32–128 samples)	Best of both — the standard in practice
Adam	Adaptive per-weight learning rates + momentum tracking	Default choice — fast, stable, robust
RMSprop	Adapts learning rate based on recent gradient magnitudes	Good for RNNs and non-stationary problems

Epochs and Batch Size

Term	Definition	Practical Guidance
Epoch	One complete pass through all training data	Use 20–50 with EarlyStopping
Batch Size	Samples processed before one weight update	32 or 128 — powers of 2 for GPU efficiency
Iteration	One batch = one forward + backward pass + update	Iterations per epoch = $N / \text{batch_size}$
Learning Rate	Step size for weight updates (default 0.001 with Adam)	Rarely need to change with Adam

Overfitting vs Underfitting — What the Loss Curves Tell You

Healthy training: Both train loss and val loss decrease and run close together. Model is learning genuine patterns.

Overfitting: Train loss keeps dropping but val loss starts rising. Model has memorised training data. Fix: add Dropout, reduce model size, get more data.

Underfitting: Both losses are high and barely decreasing. Model is too simple. Fix: add more neurons, more layers, train longer

Your First Neural Network with Keras

What is Keras?

Keras is a high-level Python library built on top of TensorFlow. It lets you build, train, and evaluate neural networks using simple, readable code — without writing the backpropagation algorithm or GPU matrix operations yourself.

The Four Keras Steps

1. Build: Stack layers with `keras.Sequential()`. Each Dense layer is a fully connected layer where every neuron connects to every neuron in the next layer.
2. Compile: Choose your loss function (what to minimise) and optimiser (how to minimise it). This is where you wire up the learning machinery.
3. Fit: Call `model.fit()` with your data. Keras runs the training loop — forward pass, loss, backprop, update — for every batch and every epoch.
4. Evaluate: Call `model.evaluate()` on test data you have never trained on. This is your true accuracy number.

Building the Model — Annotated Line by Line

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# ■ Step 1: Load MNIST
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()

# ■ Step 2: Preprocess
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0
print(f'Train: {X_train.shape} Test: {X_test.shape}')

# ■ Step 3: Build the model
model = keras.Sequential([
```



```
model = keras.Sequential([
    keras.Input(shape=(784,)),
    layers.Dense(234, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(10, activation='softmax'),
])
model.summary()
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 — 0s 0us/step
Train: (60000, 784) Test: (10000, 784)
Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 234)	183,690
dropout (Dropout)	(None, 234)	0
dense_1 (Dense)	(None, 64)	15,040
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 10)	650

Total params: 199,380 (778.83 KB)
Trainable params: 199,380 (778.83 KB)
Non-trainable params: 0 (0.00 B)

Compile and Train

```
# Step 4: Compile
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'],
)

# Step 5: Callbacks - smart training
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
early_stop = EarlyStopping(monitor='val_loss', patience=5,
    restore_best_weights=True, verbose=1)
# Halve learning rate if val_loss stalls for 3 epochs
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
    patience=3, min_lr=1e-6, verbose=1)

# Step 6: Train
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=128,
    validation_split=0.1,
    callbacks=[early_stop, reduce_lr],
    verbose=1
)
# history.history is a dict: {'loss':[], 'accuracy':[], 'val_loss':[], 'val_accuracy':[]}
```

Epoch 1/50
422/422 — 5s 8ms/step - accuracy: 0.8576 - loss: 0.4728 - val_accuracy: 0.9548 - val_loss: 0.1492 - learn
Epoch 2/50
422/422 — 3s 7ms/step - accuracy: 0.9392 - loss: 0.2098 - val_accuracy: 0.9680 - val_loss: 0.1088 - learn
Epoch 3/50
422/422 — 3s 6ms/step - accuracy: 0.9537 - loss: 0.1563 - val_accuracy: 0.9737 - val_loss: 0.0912 - learn
Epoch 4/50
422/422 — 3s 7ms/step - accuracy: 0.9611 - loss: 0.1294 - val_accuracy: 0.9772 - val_loss: 0.0767 - learn
Epoch 5/50
422/422 — 4s 9ms/step - accuracy: 0.9674 - loss: 0.1091 - val_accuracy: 0.9780 - val_loss: 0.0768 - learn
Epoch 6/50
422/422 — 3s 6ms/step - accuracy: 0.9710 - loss: 0.0951 - val_accuracy: 0.9790 - val_loss: 0.0747 - learn
Epoch 7/50
422/422 — 3s 6ms/step - accuracy: 0.9731 - loss: 0.0885 - val_accuracy: 0.9802 - val_loss: 0.0716 - learn
Epoch 8/50
422/422 — 3s 6ms/step - accuracy: 0.9774 - loss: 0.0760 - val_accuracy: 0.9807 - val_loss: 0.0655 - learn
Epoch 9/50
422/422 — 4s 9ms/step - accuracy: 0.9782 - loss: 0.0700 - val_accuracy: 0.9833 - val_loss: 0.0624 - learn
Epoch 10/50
422/422 — 3s 6ms/step - accuracy: 0.9796 - loss: 0.0653 - val_accuracy: 0.9827 - val_loss: 0.0674 - learn
Epoch 11/50
422/422 — 3s 6ms/step - accuracy: 0.9806 - loss: 0.0620 - val_accuracy: 0.9788 - val_loss: 0.0709 - learn
Epoch 12/50
420/422 — 0s 6ms/step - accuracy: 0.9827 - loss: 0.0542
Epoch 12: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
422/422 — 3s 6ms/step - accuracy: 0.9825 - loss: 0.0560 - val_accuracy: 0.9832 - val_loss: 0.0644 - learn
Epoch 13/50
422/422 — 3s 8ms/step - accuracy: 0.9851 - loss: 0.0450 - val_accuracy: 0.9833 - val_loss: 0.0619 - learn
Epoch 14/50

```

422/422 ————— 3s 8ms/step - accuracy: 0.9865 - loss: 0.0416 - val_accuracy: 0.9828 - val_loss: 0.0620 - learn
Epoch 15/50
422/422 ————— 3s 6ms/step - accuracy: 0.9867 - loss: 0.0403 - val_accuracy: 0.9833 - val_loss: 0.0637 - learn
Epoch 16/50
419/422 ————— 0s 6ms/step - accuracy: 0.9884 - loss: 0.0347
Epoch 16: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
422/422 ————— 3s 6ms/step - accuracy: 0.9876 - loss: 0.0369 - val_accuracy: 0.9827 - val_loss: 0.0641 - learn
Epoch 17/50
422/422 ————— 3s 6ms/step - accuracy: 0.9899 - loss: 0.0316 - val_accuracy: 0.9838 - val_loss: 0.0629 - learn
Epoch 18/50
422/422 ————— 4s 9ms/step - accuracy: 0.9900 - loss: 0.0311 - val_accuracy: 0.9833 - val_loss: 0.0621 - learn
Epoch 18: early stopping
Restoring model weights from the end of the best epoch: 13.

```

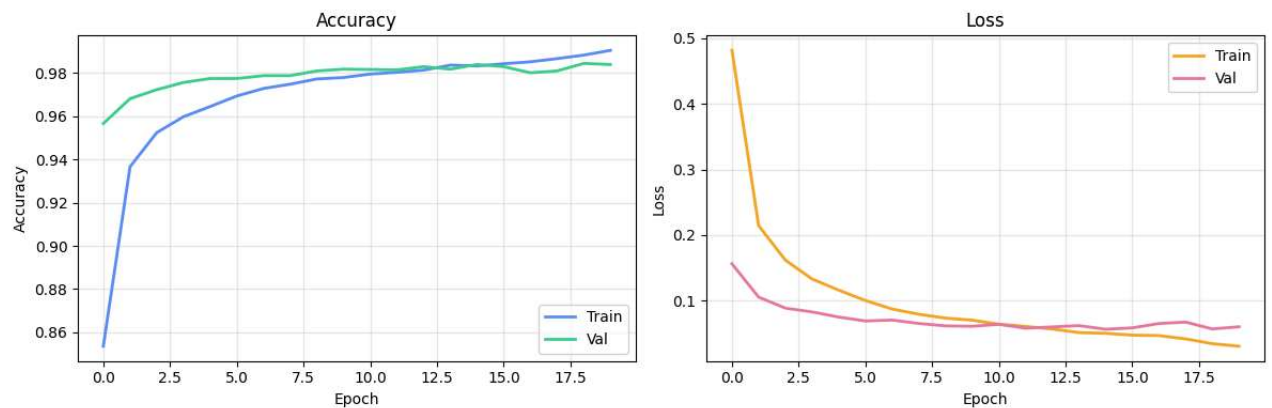
Evaluate and Plot

```

# ■ Step 7: Evaluate on test set
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f'Test Accuracy: {test_acc*100:.2f}%') # expect 97-98%
print(f'Test Loss: {test_loss:.4f}')
# ■ Step 8: Plot training curves
import matplotlib.pyplot as plt
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
# Accuracy
ax1.plot(history.history['accuracy'], label='Train', color='#5B8FF9', lw=2)
ax1.plot(history.history['val_accuracy'], label='Val', color='#3ECF8E', lw=2)
ax1.set(title='Accuracy', xlabel='Epoch', ylabel='Accuracy')
ax1.legend(); ax1.grid(alpha=0.3)
# Loss
ax2.plot(history.history['loss'], label='Train', color='#F5A623', lw=2)
ax2.plot(history.history['val_loss'], label='Val', color='#E879A0', lw=2)
ax2.set(title='Loss', xlabel='Epoch', ylabel='Loss')
ax2.legend(); ax2.grid(alpha=0.3)
plt.tight_layout(); plt.show()
# Reading curves:
# GOOD: both curves decline and stay close together
# OVERFIT: train keeps dropping, val starts rising
# UNDERFIT: both stay high — model too simple or not trained enough

```

Test Accuracy: 98.11%
Test Loss: 0.0677



Theory Deep Dive

The Vanishing Gradient Problem

During backpropagation, gradients flow backward through every layer using the chain rule. Each layer multiplies the gradient by a number. With sigmoid activations, that number is always between 0 and 0.25 (because the sigmoid derivative never exceeds 0.25).

With 10 layers: $0.25^{10} = 0.00000095$ — essentially zero. Weights in early layers receive a gradient so small they barely update. The network's front end stops learning entirely. This was the main barrier to training deep networks before 2010

Solutions to Vanishing Gradient

ReLU Activation: Gradient of ReLU is 1 for positive inputs. No shrinking. This single change enabled deep networks to be trained reliably.

Batch Normalisation: Normalises layer inputs during training so gradients stay in a healthy range throughout the network depth.

Residual Connections (ResNet): Skip connections allow gradients to flow directly backward bypassing layers, preventing the gradient from being multiplied to zero.

The Universal Approximation Theorem

A neural network with one hidden layer and enough neurons can approximate any continuous function to any desired accuracy. This is the mathematical guarantee that neural networks are not limited — they can in principle learn any mapping from inputs to outputs.

In practice: one wide layer requires exponentially many neurons for complex functions. Deep networks (many layers, moderate width) achieve the same using hierarchical composition — each layer builds on the previous, reusing simpler patterns to represent complex ones. This is why depth matters more than width.

Parameter Count — Understanding model.summary()

Layer	Input Size	Neurons	Parameters Formula	Count
Dense (128)	784	128	$784 \times 128 + 128$ bias	100,480
Dropout (0.2)	128	128	0 — no learnable params	0
Dense (64)	128	64	$128 \times 64 + 64$ bias	8,256
Dropout (0.2)	64	64	0 — no learnable params	0
Dense (10)	64	10	$64 \times 10 + 10$ bias	650
TOTAL				109,386

Regularisation Techniques

Technique	How It Works	Keras Code
Dropout	Randomly sets a fraction of neurons to zero during each training step, reducing overfitting.	<code>Dropout(0.2)</code>
L2 / Weight Decay	Adds a penalty for large weights to the loss function, encouraging simpler models.	<code>Dense(64, kernel_regularizer='l2')</code>
Early Stopping	Stops training when validation loss stops improving, preventing overfitting.	<code>EarlyStopping(patience=5)</code>
Batch Normalization	Normalizes layer inputs, stabilizes learning, and often speeds up training.	<code>BatchNormalization()</code>
Data Augmentation	Artificially expands the dataset using transformations like flips, rotations, zooms, etc.	<code>ImageDataGenerator(...)</code>
More Data	The most effective regularization technique; provides the model with more examples to learn from.	Collect or generate more training samples

Hyperparameter Quick Reference

Hyperparameter	Start Here	Too Low →	Too High →
Learning Rate	0.001 (Adam)	Training very slow	Loss explodes or oscillates
Batch Size	128	Slow, noisy updates	Less generalisation, more memory
Neurons per Layer	64–256	Underfitting	Overfitting, slower training
Dropout Rate	0.2–0.3	Overfitting persists	Underfitting, slow convergence
Number of Layers	2 hidden	Underfitting	Vanishing gradients, overfitting

Neural Networks vs Classical ML

Use Case	Best Choice	Why
----------	-------------	-----